深度学习详解（Datawhale、李宏毅）——第 1 章 机器学习基础

机器学习（Machine Learning，ML），顾名思义，机器具备有学习的能力。具体来讲，机器学习就是让机器具备找一个函数的能力。

随着要找的函数不同，机器学习有不同的类别。假设要找的函数的输出是一个数值，一个
标量（scalar），这种机器学习的任务称为**回归（regression）**。

除了回归以外，另一个常见的任务是**分类（classification）**。分类任务要让机器做选择题。人类先准备好一些选项，这些选项称为**类别（class）**，现在要找的函数的输出就是从设定好的选项里面选择一个当作输出，该任务称为分类。例如，AlphaGo 也是一个分类的问题。

在机器学习领域里面，除了回归跟分类以外，还有**结构化学习（structured learning）**。机器不只是要做选择题或输出一个数字，而是产生一个有结构的物体，比如让机器画一张图，写一篇文章。这种叫机器产生有结构的东西的问题称为结构化学习。

1.1 案例学习

机器学习找函数的过程，分成 3 个步骤。第一个步骤是写出一个带有未知参数的函数 f，比如将函数写成

$$y = b + wx_1$$

机器学习就需要一些**领域知识（domain knowledge）**，即对这个问题本质上的了解。这对于未知参数的猜测是有作用的。带有未知的**参数（parameter）**的函数称为**模型（model）**。模型在机器学习里面，就是一个带有未知的参数的函数，**特征（feature）** x1 是这个函数里面已知的，而 w 跟 b 是未知的参数。w 称为**权重（weight）**，b 称为**偏置（bias）**。这个是第一个步骤。真实值称为**标签（label）**。

第 2 个步骤是定义**损失（loss）**，损失也是一个函数，记作 $L(b, w)$。损失函数输出的值意味着，当把模型参数设定为某个数值时，这个数值好还是不好。

估测的值跟实际的值之间的差距，其实有不同的计算方法，计算 y 与 $\hat{y}$ 之间绝对值的差距，称为**平均绝对误差（Mean Absolute Error，MAE）**。

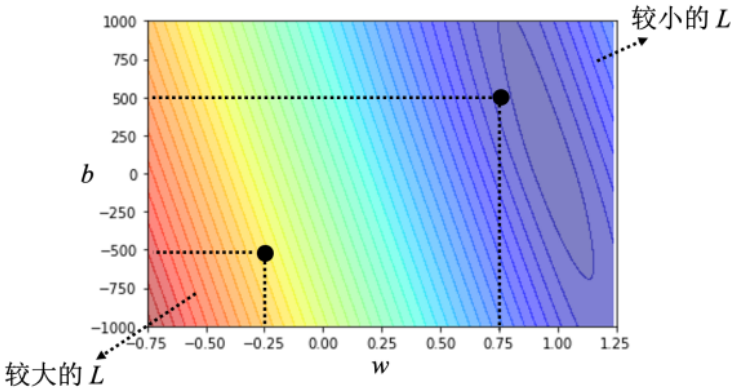
$$e = |\hat{y} - y|$$

如果算 y 与 $\hat{y}$ 之间平方的差距，则称为**均方误差（Mean Squared Error，MSE）**。

$$e = (\hat{y} - y)^2$$

有一些任务中 y 与 $\hat{y}$ 都是概率分布，这个时候可能会选择**交叉熵（cross entropy）**。

下图所示的等高线图，就是试了不同的参数，计算它的损失，画出来的等高线图称为**误差表面（error surface）**。这是机器学习的第 2 步。



接下来进入机器学习的第 3 步：解一个最优化的问题。找一个 w 跟 b，把未知的参数找一个数值出来，看代哪一个数值进去可以让损失 L 的值最小，就是要找的 w 跟 b。这个可以让损失最小的 w 跟 b 称为 w^* 跟 b^* 代表它们是最好的一组 w 跟 b，可以让损失的值最小。

梯度下降（gradient descent）是经常会使用优化的方法。步伐的大小取决于两件事情：

- 第一件事情是这个地方的斜率，斜率大步伐就跨大一点，斜率小步伐就跨小一点。
- 另外，**学习率（learning rate）** η 也会影响步伐大小。学习率是自己设定的。这种在做机器学习，需要自己设定，不是机器自己找出来的，称为**超参数（hyperparameter）**。

> Q: 为什么损失可以是负的?
> A: 损失函数是自己定义的，比如设置一个损失函数为绝对值再减 100，其可能就有负的。这个曲线并不是一个真实的损失，并不是一个真实任务的误差表面。因此这个损失的曲线可以是任何形状。

第一步参数更新：

$$w^1 \leftarrow w^0 - \eta \left.\frac{\partial L}{\partial w}\right|_{w=w^0}$$

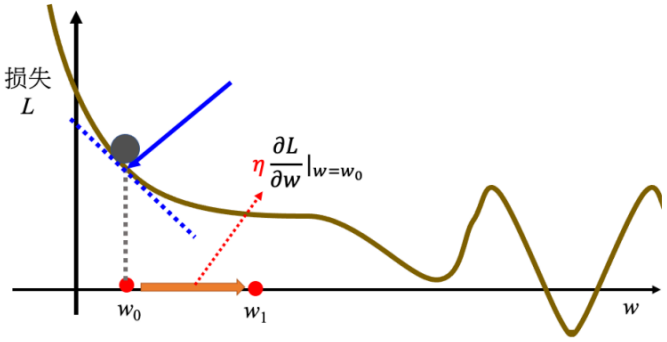


图 1.3　优化过程

再继续反复做同样的操作，不断地移动 w 的位置，最后会停下来。往往有两种情况会停下来。

- 第一种情况是一开始会设定说，在调整参数的时候，在计算微分的时候，最多计算几次。上限可能会设为 100 万次，参数更新 100 万次后，就不再更新了，更新次数也是一个超参数。
- 还有另外一种理想上的，停下来的可能是，当不断调整参数，调整到一个地方，它的偏导数是 0 。此时哪怕继续迭代，参数的位置也不再更新。

梯度下降有一个很大的问题，没有找到真正最好的解，没有找到可以让损失最小的 w 。右侧红点这个位置是真的可以让损失最小的地方，称为**全局最小值（global minimum）**，而 w^T 这个地方称为**局部最小值（local minimum）**，其左右两边都比这个地方的损失还要高一点，但是它不是整个误差表面上面的最低点。

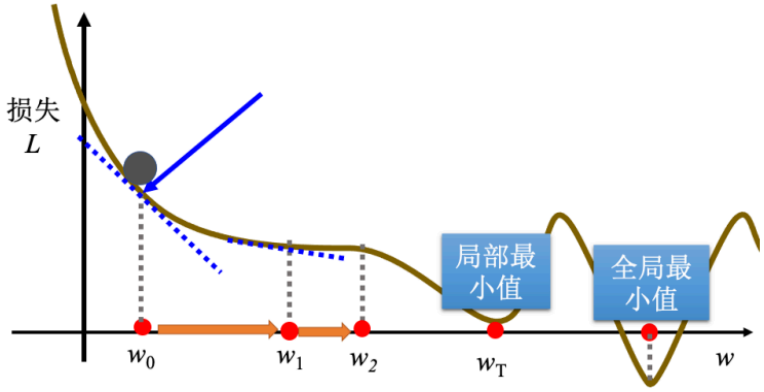


图 1.4　局部最小值

所以常常可能会听到有人讲到梯度下降不是个好方法，这个方法会有局部最小值的问题，无法真的找到全局最小值。事实上局部最小值是一个假问题，在做梯度下降的时

候，真正面对的难题不是局部最小值。有两个参数的情况下使用梯度下降，其实跟刚才一个参数没有什么不同。如果一个参数没有问题的话，可以很快的推广到两个参数。

> 因为初始值是随机选取的，所以初始值的选取好坏会决定是否会陷入局部最小值的问题。

$$
\begin{aligned}
w^1 &\leftarrow w^0 - \eta \left.\frac{\partial L}{\partial w}\right|_{w=w^0,b=b^0} \\
b^1 &\leftarrow b^0 - \eta \left.\frac{\partial L}{\partial b}\right|_{w=w^0,b=b^0}
\end{aligned}
$$

在深度学习框架里面，比如 PyTorch 里面，算微分都是程序自动帮计算的。就是反复同样的步骤，就不断的更新 w 跟 b，期待最后，可以找到一个最好的 w^* 跟最好的 b^*。

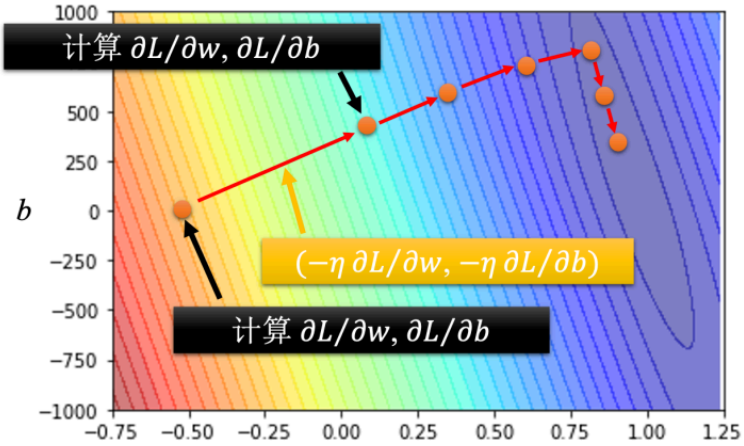


图 1.5　梯度下降优化的过程

1.2 线性模型

通常一个模型的修改，往往来自于对这个问题的理解，即领域知识。因此建模方程会出现不同，可以根据领域知识进行初步判断，然后对比性能来选择合适的模型。

例如，同样是线性模型，会有如下不同的线性模型（参数量不同）：

$$y = b + wx_1$$

$$y = b + \sum_{j=1}^{7} w_j x_j$$

$$y = b + \sum_{j=1}^{28} w_j x_j$$

这些模型都是把输入的特征 x 乘上一个权重，再加上一个偏置就得到预测的结果，这样的模型称为**线性模型（linear model）**。接下来会看如何把线性模型做得更好。

1.2.1 分段线性曲线

线性模型也许过于简单，x1 跟 y 可能中间有比较复杂的关系，如图 1.7 所示。不管如何设置w跟 b，永远制造不出红色线，永远无法用线性模型制造红色线。显然线性模型有很大的限制，这一种来自于模型的限制称为**模型的偏差**，无法模拟真实的情况。

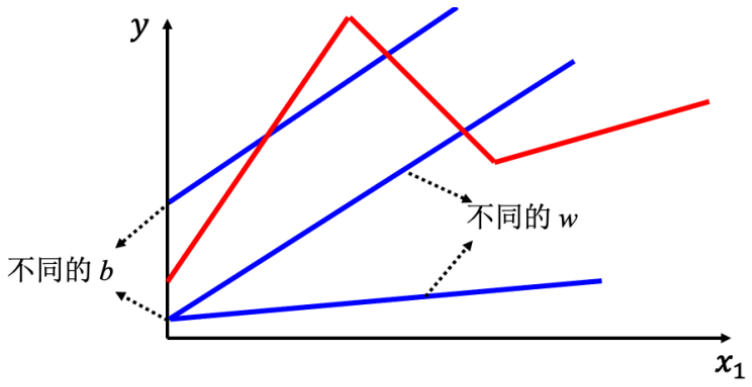


图 1.7　线性模型的局限性

所以需要写一个更复杂的、更有灵活性的、有未知参数的函数。红色的曲线可以看作是一个常数再加上一群 **Hard Sigmoid 函数**（hard sigmoid 函数的特性是当输入的值介于两个阈值间的时候，图像呈现出一个斜坡，其余位置都是水平的）。常数项设成红色的线跟 x 轴的交点一样大。蓝线 1 函数斜坡的起点，设在红色函数的起始的地方，第 2 个斜坡的终点设在第一个转角处，让第 1 个蓝色函数的斜坡和红色函数的斜坡的斜率是一样的，这个时候把 0+1 就可以得到红色曲线左侧的线段。接下来，再加第 2 个蓝色的函数，所以第 2 个蓝色函数的斜坡就在红色函数的第一个转折点到第 2 个转折点之间，让第 2 个蓝色函数的斜率跟红色函数的斜率一样，这个时候把 0+1+2，就可以得到红色函数左侧和中间的线段。接下来第 3 个部分，第 2 个转折点之后的部分，就加第 3 个蓝色的函数，第 3 个蓝色的函数坡度的起始点设的跟红色函数转折点一样，蓝色函数的斜率设的跟红色函数斜率一样，接下来把 0+1+2+3 全部加起来，就得到完整红色的线。

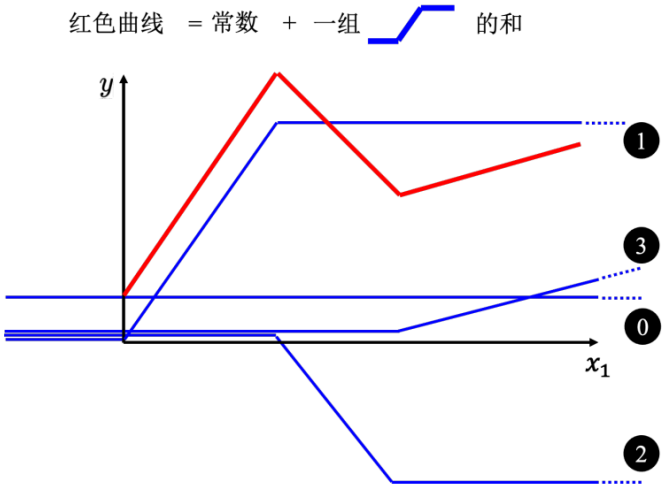


图 1.8 构建红色曲线

所以红色线，即**分段线性曲线（piecewise linear curve）**可以看作是一个常数，再加上一
堆蓝色的函数。如果分段线性曲线越复杂，转折的点越多，所需的蓝色函数就越多。

也许要考虑的 x 跟 y 的关系不是分段线性曲线，而是如图 1.9 所示的曲线。可以在这样
的曲线上面，先取一些点，再把这些点点起来，变成一个分段线性曲线。如果点取的够多或点取的位置适当，分段线性曲线就可以逼近这一个连续的曲线，就可以逼近有角度的、有弧度的这一条曲线。所以可以用分段线性曲线去逼近任何的连续的曲线，而每个分段线性曲线都可以用一大堆蓝色的函数组合起来。

> hard sigmoid 函数有如下问题：（因此我们用sigmoid函数来近似hard sigmoid）
>
> - 在两端是“平的”，导数为 0
> - 在拐点处不可导（或至少不光滑）
> - 用梯度下降时，不够“顺滑”，优化不方便（尤其在神经网络里你希望激活函数平滑可导）

> sigmoid 相比 hard sigmoid 的优势：
>
> - **处处可导、很平滑**
> - 梯度下降更好用（反向传播需要导数）

假设 x 跟 y 的关系非常复杂也没关系，就想办法写一个带有未知数的函数。直接写 Hard Sigmoid 不是很容易，但是可以用一条曲线来理解它，用 Sigmoid 函数（也叫 S 型函数）来逼近 Hard Sigmoid，如图 1.10 所示。Sigmoid 函数的表达式为

$$y = c \cdot \frac{1}{1 + e^{-(b+wx_1)}}$$

其横轴输入是 x_1，输出是 y，c 为常数。

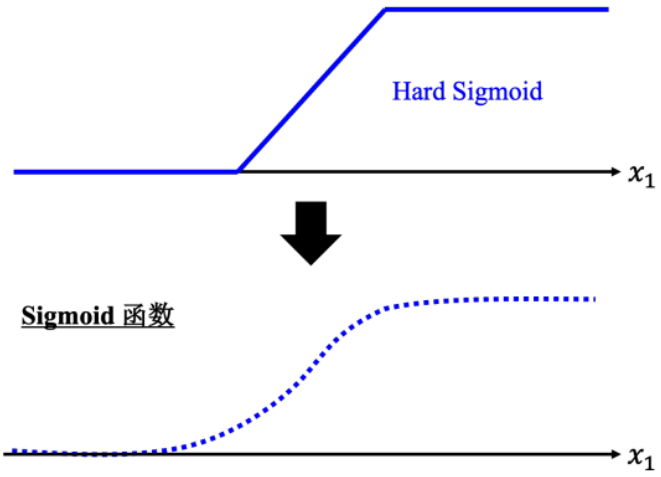


图 1.10　使用 Sigmoid 逼近 Hard Sigmoid

如果 x_1 的值，趋近于无穷大的时候，$e^{-(b+wx_1)}$ 这一项就会消失，当 x_1 非常大的时候，这一条就会收敛在高度为 c 的地方。如果 x_1 负的非常大的时候，分母的地方就会非常大，y 的值就会趋近于 0。

为了简洁，去掉了指数的部分，表达式为

$$y = c\sigma\left(b + wx_1\right)$$

所以可以用 Sigmoid 函数逼近 Hard Sigmoid 函数。

调整这里的 b、w 和 c 可以制造各种不同形状的 Sigmoid 函数，用各种不同形状的 Sigmoid 函数去逼近 Hard Sigmoid 函数。

如图 1.11 所示，如果改 w，就会改变斜率，就会改变斜坡的坡度。如果改了 b，就可以把这一个 Sigmoid 函数左右移动；如果改 c，就可以改变它的高度。所以只要有不同的 w 不同的 b 不同的 c，就可以制造出不同的 Sigmoid 函数，把不同的 Sigmoid 函数叠起来以后就可以去逼近各种不同的分段线性函数；分段线性函数可以拿来近似各种不同的连续的函数。

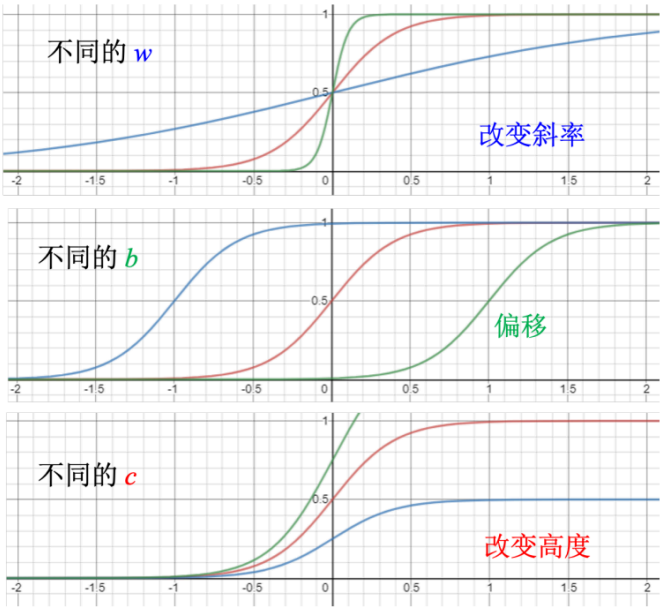


图 1.11　调整参数，制造不同的 Sigmoid 函数

如图 1.12 所示，红色这条线就是 0 加 1+2+3，而 1、2、3 都是蓝色的函数，其都可写
成$(b+wx_1)$，去做 Sigmoid 再乘上 c_i，只是 1、2、3 的 w、b、c 不同。

$$y=b+\sum_i c_i\sigma\left(b_i+w_ix_1\right)$$

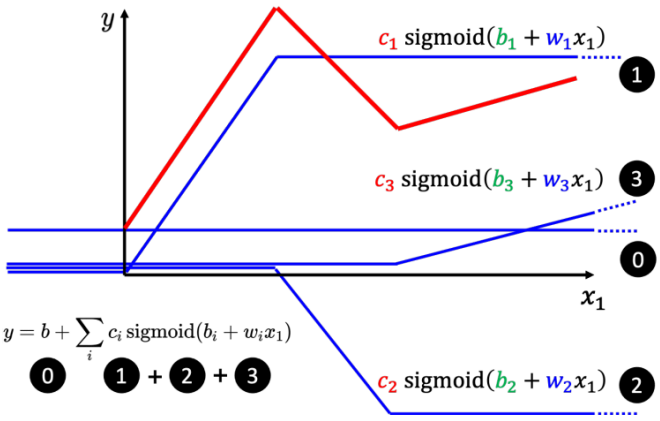


图 1.12　使用 Hard Sigmoid 来合成红色

此外，我们可以不只用一个特征 x_1，可以用多个特征代入不同的 c, b, w，组合出各种不
同的函数，从而得到更有**灵活性（flexibility）**的函数，如图 1.13 所示。

> 当有多个特征时，要把“分段线性曲线”的直觉从 **1D 曲线**升级成 **多维空间里的分段线性“曲面/超曲面”**

> 此时的 Sigmoid 函数是一个超平面：
>
> - 在这个超平面的一侧，Sigmoid 输出接近 0
> - 另一侧接近 1
> - 过渡区是一条（更准确说：一片）“薄层”，围绕这张超平面
>
> 所以，多维情况下你仍然可以用“很多个平滑台阶叠加”的方式理解，只是：
>
> - 1D：台阶发生在 **点附近**（阈值位置），拼出分段线性“曲线”
> - 多D：台阶发生在 **超平面附近**，拼出分段线性“曲面/分区边界”
>
> 总结：
>
> **输入空间被很多超平面切成许多区域，每个区域里模型的行为可以近似成比较简单的形状；区域与区域之间通过 Sigmoid 的平滑过渡连接起来。**

$$y = \underline{b + wx_1}$$

$$y = b + \sum_i c_i \,\mathrm{sigmoid}(\underline{b_i + w_i x_1})$$

$$y = b + \underline{\sum_j w_j x_j}$$

$$\boxed{y = b + \sum_i c_i \,\mathrm{sigmoid}\left(\underline{b_i + \sum_j w_{ij} x_j}\right)}$$

图 1.13　构建更有灵活性的函数

直观来讲，先考虑一下 j 就是 1、2、3 的情况，就是只考虑 3 个特征。每一个 i 就代表了一个蓝色的函数。每一个蓝色的函数都用一个 Sigmoid 函数来比近似它，1,2,3 代表有个 Sigmoid 函数。

w_{ij} 代表在第 i 个 Sigmoid 里面，乘给第 j 个特征的权重，w 的第一个下标代表是现在在考虑的是第几个 Sigmoid 函数。为了简化起见，括号里面的式子为

$$
\begin{aligned}
r_1 &= b_1 + w_{11}x_1 + w_{12}x_2 + w_{13}x_3 \\
r_2 &= b_2 + w_{21}x_1 + w_{22}x_2 + w_{23}x_3 \\
r_3 &= b_3 + w_{31}x_1 + w_{32}x_2 + w_{33}x_3
\end{aligned}
$$

我们可以用矩阵跟向量相乘的方法，写一个比较简洁的写法。

$$
\begin{bmatrix} r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}
$$

将其改成线性代数比较常用的表示方式为

$$
\boldsymbol{r} = \boldsymbol{b} + \boldsymbol{W}\boldsymbol{x}
$$

蓝框里面的括号里面做的如式 (1.21) 所示，r 对应的是 r_1, r_2, r_3。 r_1, r_2, r_3 分别通过 Sigmoid 函数得到 a_1, a_2, a_3，即

$$
\boldsymbol{a} = \sigma(\boldsymbol{r})
$$

上面这个比较有灵活性的函数，如果用线性代数来表示，即

$$
y = b + \boldsymbol{c}^{\mathrm{T}}\boldsymbol{a}
$$

接下来，如图 1.15 所示，x 是特征，绿色的 $\boldsymbol{b}$ 是一个向量，灰色的 b 是一个数值。$\boldsymbol{W}, \boldsymbol{b}, \boldsymbol{c^T}, b$ 是未知参数。把这些东西通通拉直，“拼”成一个很长的向量，我们把 W 的每一行或者是每一列拿出来。这个长的向量直接用 θ 来表示。所有的未知的参数，一律统称 θ。

> Q: 优化是找一个可以让损失最小的参数，是否可以穷举所有可能的未知参数的值?
> A：只有 w 跟 b 两个参数的前提之下，可以穷举所有可能的 w 跟 b 的值，所以在参数
> 很少的情况下。甚至可能不用梯度下降，不需要优化的技巧。但是参数非常多的时候，
> 就不能使用穷举的方法，需要梯度下降来找出可以让损失最低的参数。

> Q：刚才的例子里面有 3 个 Sigmoid，为什么是 3 个，能不能 4 个或更多?
> A：Sigmoid 的数量是由自己决定的，而且 Sigmoid 的数量越多，可以产生出来的分段
> 线性函数就越复杂。Sigmoid 越多可以产生有越多段线的分段线性函数，可以逼近

> 越复
> 杂的函数。**Sigmoid 的数量也是一个超参数。**

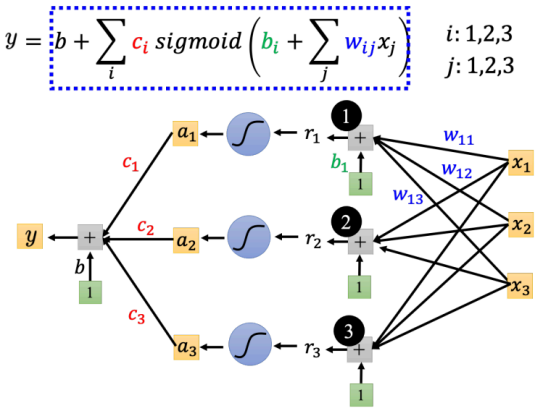


图 1.14　比较有灵活性函数的计算过程

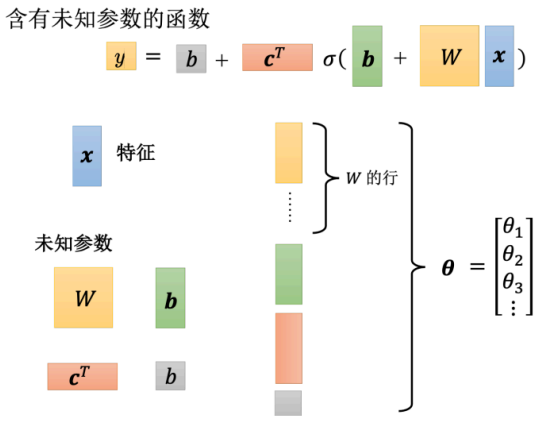


图 1.15　未知参数“拼”成一个向量

接下来要定义损失。用 θ 来统设所有的参数，所以损失函数就变成 $L(\theta)$。损失函数能够判断 θ 的好坏，其计算方法跟刚才只有两个参数的时候是一样的。

先给定 θ 的值，即某一组 $\boldsymbol{W}, \boldsymbol{b}, \boldsymbol{c}^{\boldsymbol{T}}, b$ 的值，再把一种特征 $\boldsymbol{x}$ 代进去，得到估测出来的 y，再计算一下跟真实的标签之间的误差 e。把所有的误差通通加起来，就得到损失。

接下来下一步就是优化

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \end{bmatrix}$$

要找到 $\boldsymbol{\theta}$ 让损失越小越好，可以让损失最小的一组 $\boldsymbol{\theta}$ 称为 $\boldsymbol{\theta}^*$。一开始要随机选一个初始的数值 $\boldsymbol{\theta}^{\boldsymbol{0}}$。接下来计算每一个未知的参数对 L 的微分，得到向量 $\boldsymbol{g}$，即可以让损失变低的函数

$$\boldsymbol{g} = \nabla L(\boldsymbol{\theta}_0)$$

$$\boldsymbol{g} = \begin{bmatrix} \left.\frac{\partial L}{\partial \theta_1}\right|_{\boldsymbol{\theta}=\boldsymbol{\theta}_0} \\ \left.\frac{\partial L}{\partial \theta_2}\right|_{\boldsymbol{\theta}=\boldsymbol{\theta}_0} \\ \vdots \end{bmatrix}$$

这个向量也称为梯度，∇L 代表梯度。计算出 $\boldsymbol{g}$ 后，接下来更新参数

$$\begin{bmatrix}\theta_1^1\\ \theta_1^2\\ \vdots\end{bmatrix} \leftarrow \begin{bmatrix}\theta_0^1\\ \theta_0^2\\ \vdots\end{bmatrix} - \begin{bmatrix}\eta\frac{\partial L}{\partial\theta_1}\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_0}\\ \eta\frac{\partial L}{\partial\theta_2}\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_0}\\ \vdots\end{bmatrix}$$

$$\boldsymbol{\theta}_1 \leftarrow \boldsymbol{\theta}_0 - \eta\boldsymbol{g}$$

以此类推，直到不想做。或者计算出梯度为 $\mathbf{0}$ 向量，导致无法再更新参数为止，不过在实现上几乎不太可能梯度为 $\mathbf{0}$，通常会停下来就是我们不想做了。

但实现上有个细节的问题，实际使用梯度下降的时候，如图 1.17 所示，会把 N 笔数据随机分成一个一个的**批量（batch）**，一组一组的。每个批量里面有 B 笔数据，所以本来有 N 笔数据，现在 B 笔数据一组，一组叫做批量。本来是把所有的数据拿出来算一个损失，现在只拿一个批量里面的数据出来算一个损失，记为 L_1 跟 L 以示区别。所以实现上每次会先选一个批量，用该批量来算 L_1，根据 L_1 来算梯度，再用梯度来更新参数，接下来再选下一个批量算出 L_2，根据 L_2 算出梯度，再更新参数，再取下一个批量算出 L_3，根据 L_3 算出梯度，再用 L_3 算出来的梯度来更新参数。

所以并不是拿 L 来算梯度，实际上是拿一个批量算出来的 L_1, L_2, L_3 来计算梯度。把所有的批量都看过一次，称为一个**回合（epoch）**，每一次更新参数叫做一次**更新**。更新跟回合是不同的东西。每次更新一次参数叫做一次更新，把所有的批量都看过一遍，叫做一个回合。

批量大小（batch size）和 sigmoid 函数的个数都是超参数。

1.2.2 模型变形

其实还可以对模型做更多的变形，不一定要把 Hard Sigmoid 换成 Soft Sigmoid。Hard Sigmoid 可以看作是两个**修正线性单元（Rectified Linear Unit，ReLU）**的加总，ReLU 的图像有一个水平的线，走到某个地方有一个转折的点，变成一个斜坡，其对应的公式为

$$c \cdot \max(0, b + wx_1)$$

通过 w, b, c 可以挪动其位置和斜率。把两个 ReLU 叠起来就可以变成 Hard 的 Sigmoid，想要用 ReLU，就把 Sigmoid 的地方，换成 $max(0, b_i + w_{ij}x_j)$。

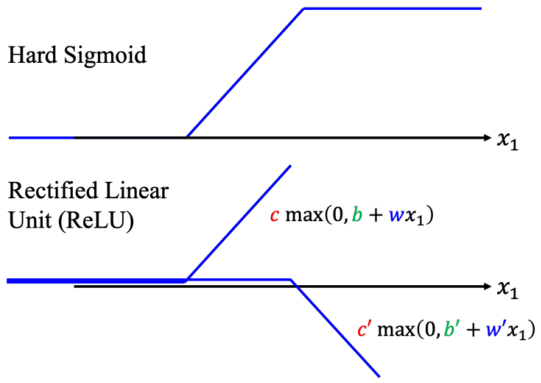


图 1.18　ReLU 函数

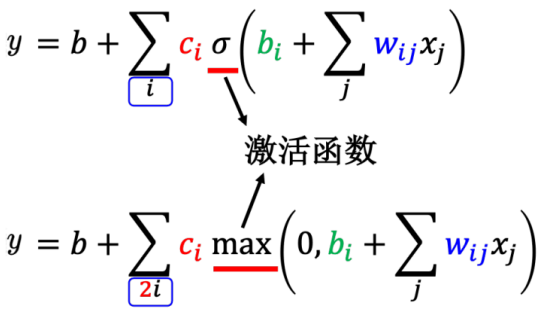


图 1.19　激活函数

如图 1.19 所示，2 个 ReLU 才能够合成一个 Hard Sigmoid。要合成 i 个 Hard Sigmoid，需要 i 个 Sigmoid，如果 ReLU 要做到一样的事情，则需要 $2i$ 个 ReLU。因此表示一个 Hard 的 Sigmoid 不是只有一种做法。在机器学习里面，Sigmoid 或 ReLU 称为**激活函数（activation function）**。

当然还有其他常见的激活函数，但 Sigmoid 跟 ReLU 是最常见的激活函数，接下来的实验都选择用了 ReLU，显然 ReLU 比较好。

> ReLU 在深度网络里通常比 sigmoid “更好用”，主要是因为：
> **1) 梯度消失：sigmoid 更容易“饱和”**
>
> - sigmoid 的输出在 (0,1)。当输入 x 很大或很小时，$\sigma(x)$ 会非常接近 1 或 0，进入**饱和区**。
> - 在饱和区里 sigmoid 的导数接近 0。
> - 深层网络反向传播时，梯度要层层相乘；如果每层都乘上一个接近 0 的数，梯度很快就“没了”（**vanishing gradient**），前面的层几乎学不到东西。
>
> 对比：
>
> - ReLU 在 (x>0) 时导数为 w（不衰减），所以在“激活”的区域，梯度传得更顺，深层网络更容易训练。
>
> **2) 计算更简单，训练更快**
>
> - sigmoid 需要指数运算，相对更“重”。
> - ReLU 就是一个 max，计算非常轻量，通常训练更快（尤其在大模型/大数据时差异更明显）。
>
> **3) 稀疏激活（sparsity）带来一些经验优势**

- ReLU 在 $x \leq 0$ 时输出 0，很多神经元在某些输入下会“关掉”，产生**稀疏激活**。
- 稀疏激活常常带来一些好处：表示更有选择性，某些情况下更不容易过拟合（经验上常见，但不一定对所有任务都绝对成立）。

4) 但 ReLU 也不是完美：会有“死亡 ReLU”

- 如果某个神经元的输入长期落在 $x \leq 0$ ，梯度为 0，就可能再也激活不起来（**dying ReLU**）。
- 常见缓解方法：更好的初始化、合适学习率、使用 Leaky ReLU / PReLU / ELU / GELU 等变体、加 BatchNorm 等。

可知，使用的 ReLU 个数越多，即宽度越高，拟合的程度越高，越接近曲线（但要注意过拟合）。除了增加宽度，也可以增加深度。继续改模型，如图 1.21 所示。不一定要通过 Sigmoid 函数，通过 ReLU 也可以得到 a，同样的事情再反复地多做几次。所以可以把 x 做这一连串的运算产生 a，接下来把 a 做这一连串的运算产生 a'。**反复地多做的次数**又是另外一个超参数。注意，w, b 和 w', b' 不是同一个参数，是增加了更多的未知的参数。

	线性	10 ReLU	100 ReLU	1000 ReLU
2017 – 2020	320	320	280	270
2021	460	450	430	430

图 1.20　激活函数实验结果

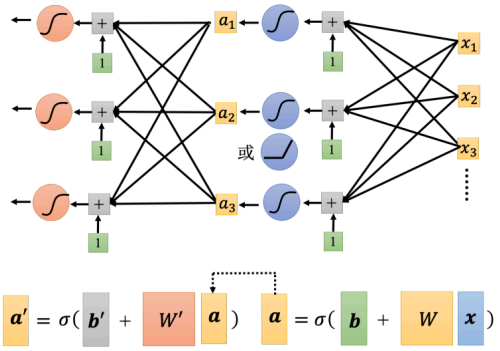


图 1.21　改进模型

如图 1.24 所示，Sigmoid 或 ReLU 称为**神经元（neuron）**，很多的神经元称为**神经网络（neural network）**。图 1.24 中每一列神经元称为神经网络的一层，又称为**隐藏层（hidden layer）**，很多的隐藏层就“深”，这套技术称为深度学习。

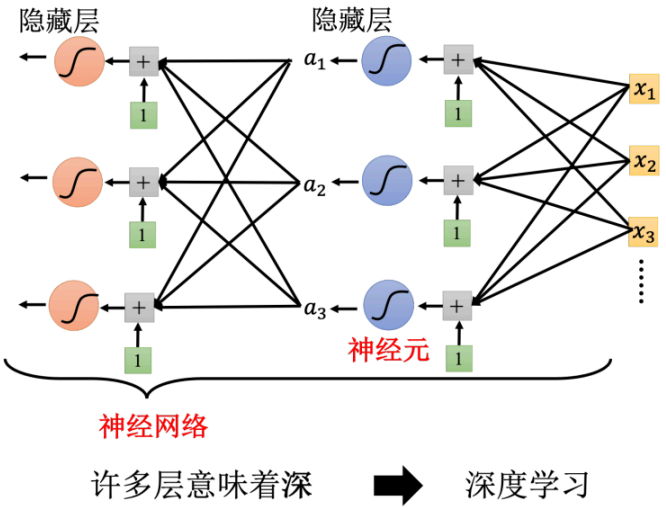


图 1.24　深度学习的结构

神经网路正向越来越深的方向发展。2012 年的 AlexNet 有 8 层它的错误率是 16.4%，两年之后 VGG 有 19 层，错误率在图像识别上进步到 7.3 %。这都是在图像识别上一个基准的数据库（ImageNet）上面的结果，后来 GoogleNet 有 22 层，错误率降到 6.7%。而**残差网络（Residual Network，ResNet）**有 152 层，错误率降到 3.57%。

在训练数据和测试数据上的结果是不一致的，这种情况称为**过拟合（overfitting）**。

深度学习的训练会用到**反向传播（BackPropagation，BP）**，其实它就是比较有效率、算梯度的方法。